

JAVA SYSTEM PROJECT

Using Java Swing, AWT, JDBC, and Apache Derby

1. PROJECT OVERVIEW

The project requires students to design and develop a **Java-based desktop information system** using the following technologies:

- **Java**
- **Java Swing** – for the graphical user interface (GUI)
- **Java AWT** – for GUI components, layouts, events, and supporting interface functions
- **JDBC (Java Database Connectivity)** – for connecting the Java application to the database
- **Apache Derby** – as the relational database management system (RDBMS)

The system must demonstrate the ability to create a functional desktop application that can:

1. Accept and validate user input.
2. Store information in a relational database.
3. Retrieve information from the database.
4. Update existing records.
5. Delete records when permitted.
6. Search and filter records.
7. Display database records in a user-friendly interface.
8. Establish relationships between multiple database tables.
9. Use proper database connectivity through JDBC.
10. Apply appropriate error handling and input validation.

The project should demonstrate both **software development** and **database management** concepts.

2. PROJECT OBJECTIVES

At the end of the project, the student should be able to:

General Objective

Develop a functional Java desktop information system that uses Swing/AWT for its graphical user interface and JDBC with Apache Derby for database operations.

Specific Objectives

The system should allow the user to:

- Log in to the system when authentication is required.
- Add new records.
- View existing records.
- Search for specific records.
- Update existing information.
- Delete records when authorized.
- Generate or display useful reports.
- Maintain related information across multiple tables.
- Validate required fields.
- Prevent invalid database entries.
- Handle database connection errors.
- Provide clear success and error messages.
- Maintain data consistency through primary keys and foreign keys.

3. SYSTEM INTRODUCTION

The project must begin with an introduction explaining the purpose and background of the **assigned system**.

The introduction should answer the following questions:

3.1 What is the system?

Provide a clear description of the system.

Example:

The **assigned system** is a Java-based desktop information management system designed to computerize the recording, monitoring, searching, and management of information that is normally handled manually.

3.2 What problem does the system solve?

Explain the existing problem or limitation.

Possible problems include:

- Manual record keeping
- Paper-based documentation
- Duplicate records
- Difficulty searching for information
- Lost or damaged records
- Slow processing
- Difficulty generating reports

- Lack of centralized information
- Human errors during data entry

3.3 Who will use the system?

Identify the intended users.

Examples:

- Administrator
- Staff
- Manager
- Employee
- Customer
- Student
- Librarian
- Cashier
- Registrar

3.4 What are the benefits?

Explain how the proposed system improves the existing process.

Examples:

- Faster record retrieval
- Improved data organization
- Reduced duplicate records
- Easier reporting
- Better data accuracy
- Improved record security
- Easier monitoring
- Centralized database storage

4. PROPOSED SYSTEM

Provide a detailed description of how the proposed system will work.

The description should explain:

1. Who enters the information.
2. What information is entered.
3. Where the information is stored.
4. How users search for information.
5. How records are updated.
6. How records are deleted.
7. What reports can be generated.
8. What users can and cannot access.

The proposed system should replace or improve the identified manual process.

5. SYSTEM SCOPE

The scope defines what the system **will include**.

The project should clearly identify the functions that are included in the system.

Example:

Included Features

- User login
- User management
- Customer management
- Product management
- Transaction management
- Search functionality
- Record management
- Database management
- Report viewing
- Logout

Excluded Features

The project should also identify features that are outside the scope.

Examples:

- Online payment
- Mobile application
- Cloud synchronization
- SMS notification
- Online customer access

The scope should prevent the project from becoming unnecessarily large.

6. SYSTEM USERS

Define the different types of users.

Example:

User Type	Description
Administrator	Has access to system configuration and user management
Staff	Can create and update operational records
Manager	Can view records and reports
Customer	May have limited access depending on the system

The actual user roles should depend on the chosen system.

7. FUNCTIONAL REQUIREMENTS

The system must contain appropriate functional requirements.

7.1 User Authentication

The system should provide:

- Username field
- Password field
- Login button
- Clear/reset button
- Exit button
- Login validation
- Error message for incorrect credentials

Example:

The system shall verify the username and password against the user database before allowing access to the main system.

7.2 Add Record

The system should allow authorized users to create new records.

Requirements:

- Input fields
- Required-field validation
- Save button
- Clear button
- Database insertion
- Success message
- Error message

7.3 View Records

The system should display stored records.

A JTable should preferably be used to display database records.

The table should support:

- Column headings
- Multiple rows
- Record selection
- Refreshing
- Scrolling

7.4 Search Records

The system should provide a search function.

Possible search fields:

- ID
- Name
- Category
- Date
- Status
- Reference number

Example:

The system shall allow users to search for records using a keyword and display matching records in the JTable.

7.5 Update Record

Users with appropriate permissions should be able to:

1. Select an existing record.
2. Load the information into input fields.
3. Modify the information.
4. Click Update.
5. Save the changes to the database.

7.6 Delete Record

The system may provide a delete function.

Before deletion, the system should display a confirmation dialog.

Example:

Are you sure you want to delete this record?

The system should not immediately delete the record without confirmation.

7.7 Clear Form

A Clear button should reset the input fields.

Example:

[Save] [Update] [Delete] [Clear]

7.8 Logout

The system should provide a logout option.

When the user logs out:

1. The current session should end.
2. The login screen should be displayed again.

8. GRAPHICAL USER INTERFACE REQUIREMENTS

The GUI must be developed using **Java Swing and AWT**.

Recommended Swing components include:

- JFrame
- JPanel
- JLabel
- JTextField
- JPasswordField
- JButton
- JComboBox
- JRadioButton
- JCheckBox
- JTable
- JScrollPane
- JTextArea
- JMenuBar
- JMenu
- JMenuItem
- JOptionPane

AWT classes may be used for:

- Layout management

- Events
- Colors
- Fonts
- Dimensions
- Dialog-related functionality

Examples:

FlowLayout
BorderLayout
GridLayout
GridBagLayout
Color
Font
Dimension
ActionListener
MouseListener

9. REQUIRED GUI SCREENS

The project should contain appropriate screens depending on the system.

At minimum, the following are recommended:

9.1 Login Form

Suggested components:

SYSTEM LOGIN

Username: [_____]

Password: [_____]

[LOGIN] [CLEAR]

[EXIT]

9.2 Main Dashboard

The dashboard should provide navigation to the different modules.

Example:

| SYSTEM NAME Logout |

| Dashboard | Users | Records | Reports | Exit |

|

| DASHBOARD

|

| Total Users: 25

| Total Records: 350

| Today's Records: 15

|

9.3 Management Form

A management form should contain:

ID: [_____]

Name: [_____]

Category: [_____]

Status: [_____]

Date: [_____]

[SAVE] [UPDATE] [DELETE] [CLEAR]

Search: [_____] [SEARCH]

| ID | Name | Category | Status | Date |

| | | | | |

10. INPUT VALIDATION REQUIREMENTS

The system must validate user input.

Examples:

Required Fields

Do not allow empty values for required fields.

Numeric Fields

Numeric fields should only accept valid numbers.

Email

If an email field exists, validate the basic email format.

Dates

Dates must use a consistent format.

Duplicate Records

The system should prevent duplicate values for fields that must be unique.

Password

Passwords should not be displayed as normal text.

Use:

JPasswordField

11. DATABASE REQUIREMENTS

The database must use **Apache Derby**.

The database should contain multiple related tables rather than placing all information in one table.

The database should demonstrate:

- Primary keys
- Foreign keys
- Relationships
- Appropriate data types
- Constraints
- Normalization
- CRUD operations

12. DATABASE DESIGN

Before creating the database, identify the major entities in the system.

For example, a transaction-based system might contain:

1. Users
2. Customers
3. Products
4. Categories
5. Transactions
6. Transaction Details

The actual tables should be based on the assigned system project.

13. SAMPLE DATABASE STRUCTURE

The following structure may be used as a model.

13.1 USERS TABLE

Stores system users.

Field	Data Type	Key	Description
user_id	INTEGER	PK	Unique user ID
username	VARCHAR(50)	UNIQUE	Login username
password	VARCHAR(255)		User password
full_name	VARCHAR(100)		User's full name
role	VARCHAR(30)		User role
status	VARCHAR(20)		Account status

13.2 CUSTOMERS TABLE

Stores customer information.

Field	Data Type	Key	Description
customer_id	INTEGER	PK	Unique customer ID
first_name	VARCHAR(50)		First name
last_name	VARCHAR(50)		Last name
contact_no	VARCHAR(30)		Contact number
email	VARCHAR(100)		Email address
address	VARCHAR(255)		Address

13.3 CATEGORIES TABLE

Stores product/service categories.

Field	Data Type	Key	Description
category_id	INTEGER	PK	Unique category ID
category_name	VARCHAR(100)	UNIQUE	Category name
description	VARCHAR(255)		Category description

13.4 PRODUCTS TABLE

Stores products or services.

Field	Data Type	Key	Description
product_id	INTEGER	PK	Unique product ID
category_id	INTEGER	FK	References category
product_name	VARCHAR(100)		Product name
price	DECIMAL(10,2)		Product price
quantity	INTEGER		Available quantity
status	VARCHAR(20)		Product status

Relationship:

CATEGORIES



One category may contain many products.

13.5 TRANSACTIONS TABLE

Stores transaction header information.

Field	Data Type	Key	Description
transaction_id	INTEGER	PK	Transaction ID
customer_id	INTEGER	FK	Customer involved
user_id	INTEGER	FK	Staff/user who processed it
transaction_date	TIMESTAMP		Date and time
total_amount	DECIMAL(10,2)		Total transaction amount
status	VARCHAR(20)		Transaction status

13.6 TRANSACTION_DETAILS TABLE

Stores individual items included in each transaction.

Field	Data Type	Key	Description
detail_id	INTEGER	PK	Detail ID
transaction_id	INTEGER	FK	Related transaction
product_id	INTEGER	FK	Purchased product
quantity	INTEGER		Quantity purchased
unit_price	DECIMAL(10,2)		Price per unit
subtotal	DECIMAL(10,2)		Quantity × unit price

14. TABLE RELATIONSHIP DIAGRAM

The database relationship diagram should show the primary keys and foreign keys.

+-----+-----+		
	USERS	
+-----+-----+		
	PK user_id	
	username	
	password	
	full_name	
	role	
	status	
+-----+-----+		
	1	
	M	

+-----V-----+		
	TRANSACTIONS	
+-----+-----+		
	PK transaction_id	
	FK customer_id	
	FK user_id	
	transaction_date	
	total_amount	
	status	
+-----+-----+		
	1	
+-----V-----+		
	TRANSACTION_DETAILS	
+-----+-----+		
	PK detail_id	
	FK transaction_id	
	FK product_id	
	quantity	
	unit_price	
	subtotal	
+-----+-----+		
	M	
	1	

+-----V-----+		
	PRODUCTS	
+-----+-----+		
	PK product_id	
	FK category_id	
	product_name	
	price	
	quantity	
	status	
+-----+-----+		
	M	
	1	
+-----V-----+		
	CATEGORIES	
+-----+-----+		
	PK category_id	
	category_name	
	description	
+-----+-----+		

+-----+-----+		
	CUSTOMERS	
+-----+-----+		
	PK customer_id	
	first_name	
	last_name	
	contact_no	
	email	
	address	
+-----+-----+		
	1	
	M	
TRANSACTIONS		

15. RELATIONSHIP EXPLANATION

Users → Transactions

One user can process many transactions.

USERS 1 ----- MANY TRANSACTIONS

Customers → Transactions

One customer can have many transactions.

CUSTOMERS 1 ----- MANY TRANSACTIONS

Categories → Products

One category can contain many products.

CATEGORIES 1 ----- MANY PRODUCTS

Transactions → Transaction Details

One transaction can contain many transaction-detail records.

TRANSACTIONS 1 ----- MANY TRANSACTION_DETAILS

Products → Transaction Details

One product can appear in many transaction-detail records.

PRODUCTS 1 ----- MANY TRANSACTION_DETAILS

Therefore, TRANSACTION_DETAILS acts as an associative/detail table between transactions and products.

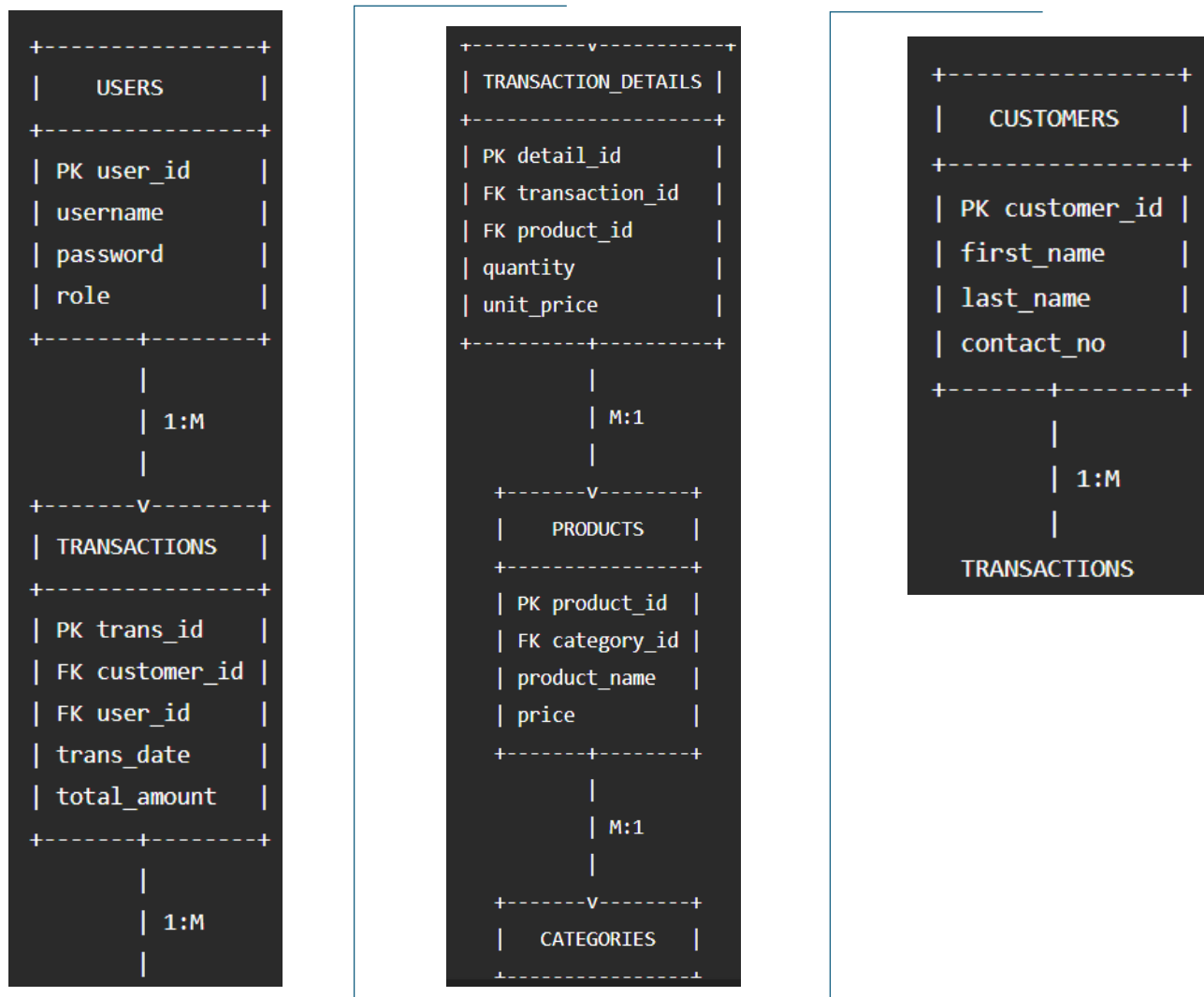
16. ENTITY RELATIONSHIP DIAGRAM (ERD)

The final documentation should include an ERD.

The ERD should show:

- Entity/table names
- Attributes
- Primary keys
- Foreign keys
- One-to-one relationships where applicable
- One-to-many relationships
- Many-to-many relationships resolved through an associative table

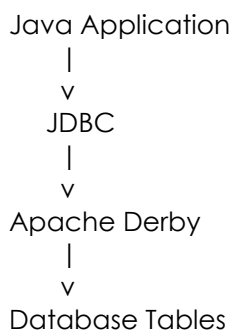
Example:



17. JDBC REQUIREMENTS

The application must use JDBC to communicate with Apache Derby.

The basic database process should be:



The application should use appropriate JDBC classes such as:

Connection
PreparedStatement
Statement
ResultSet
SQLException

18. DATABASE CONNECTION

Create a separate database connection class rather than repeating connection code throughout the project.

Example structure:

```
public class DatabaseConnection {  
  
    private static final String URL =  
        "jdbc:derby://localhost:1527/MyDatabase";  
  
    private static final String USER = "app";  
    private static final String PASSWORD = "app";  
  
    public static Connection getConnection()  
        throws SQLException {  
  
        return DriverManager.getConnection(  
            URL, USER, PASSWORD  
        );  
    }  
}
```

The actual Derby URL should match the student's database configuration.

19. PREPARED STATEMENT REQUIREMENT

Use PreparedStatement for SQL statements involving user input.

Example:

```
String sql =  
    "INSERT INTO USERS " +  
    "(username, password, full_name, role) " +  
    "VALUES (?, ?, ?, ?)";  
  
PreparedStatement ps =  
    connection.prepareStatement(sql);  
  
ps.setString(1, username);
```

```
ps.setString(2, password);  
ps.setString(3, fullName);  
ps.setString(4, role);
```

```
ps.executeUpdate();
```

This is preferable to constructing SQL statements by concatenating user input.

20. CRUD REQUIREMENTS

The system must demonstrate CRUD operations.

CRUD means:

C = CREATE

R = READ

U = UPDATE

D = DELETE

CREATE

Insert a new record.

INSERT INTO ...

READ

Retrieve records.

SELECT ...

UPDATE

Modify an existing record.

UPDATE ...

DELETE

Remove a record when authorized.

DELETE ...

21. SAMPLE CRUD SQL

INSERT

INSERT INTO CATEGORIES

(category_name, description)

VALUES (?, ?);

SELECT

SELECT *

FROM CATEGORIES

ORDER BY category_id;

UPDATE

UPDATE CATEGORIES

SET category_name = ?,

description = ?

WHERE category_id = ?;

DELETE

DELETE FROM CATEGORIES

WHERE category_id = ?;

22. JOIN REQUIREMENTS

The project should demonstrate SQL joins when information from multiple tables needs to be displayed.

Example:

```
SELECT
    p.product_id,
    p.product_name,
    c.category_name,
    p.price,
    p.quantity
FROM PRODUCTS p
INNER JOIN CATEGORIES c
    ON p.category_id = c.category_id;
```

This allows the application to display the category name instead of only displaying the category ID.

23. JAVA PROJECT STRUCTURE

The project should be organized into appropriate packages/classes.

Recommended structure:

```
src/
├── model/
│   ├── User.java
│   ├── Customer.java
│   ├── Category.java
│   ├── Product.java
│   ├── Transaction.java
│   └── TransactionDetail.java
├── dao/
│   ├── UserDAO.java
│   ├── CustomerDAO.java
│   ├── CategoryDAO.java
│   ├── ProductDAO.java
│   └── TransactionDAO.java
├── database/
│   └── DatabaseConnection.java
├── ui/
│   ├── LoginFrame.java
│   ├── DashboardFrame.java
│   ├── UserFrame.java
│   ├── CustomerFrame.java
│   ├── ProductFrame.java
│   └── TransactionFrame.java
└── Main.java
```

24. MODEL CLASSES

Model classes represent database entities.

Example:

```
public class Product {  
  
    private int productId;  
    private int categoryId;  
    private String productName;  
    private double price;  
    private int quantity;  
  
    public Product() {  
    }  
  
    // Getters and setters  
}
```

Each major database table should have an appropriate model class when using this architecture.

25. DAO CLASSES

DAO means **Data Access Object**.

DAO classes should handle database operations.

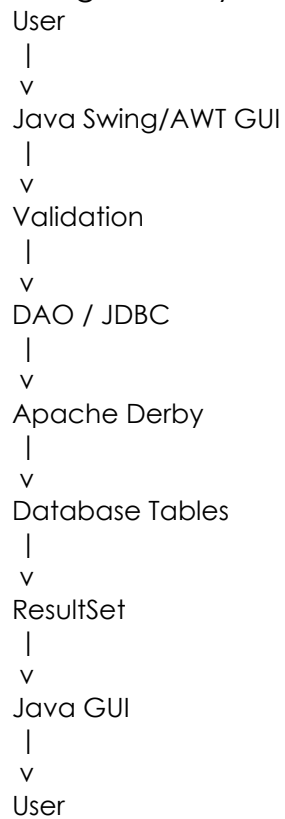
Example:

```
ProductDAO  
|  
+-- addProduct()  
+-- getProducts()  
+-- updateProduct()  
+-- deleteProduct()  
+-- searchProduct()
```

This separates database operations from the GUI.

26. USER INTERFACE AND DATABASE FLOW

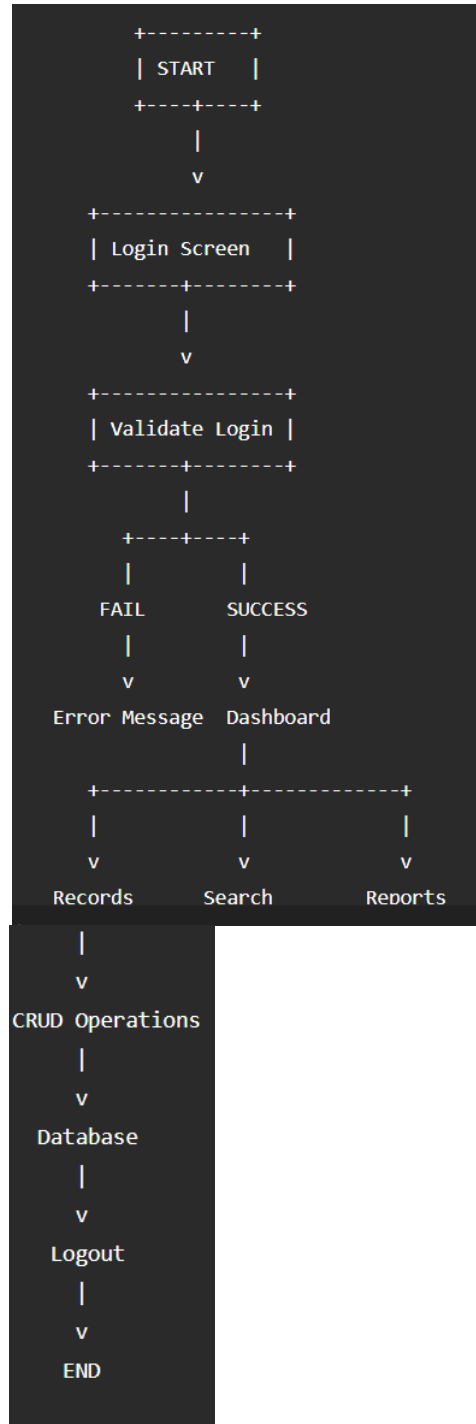
The general system flow should be:



27. SYSTEM FLOWCHART

The documentation should include a system flowchart.

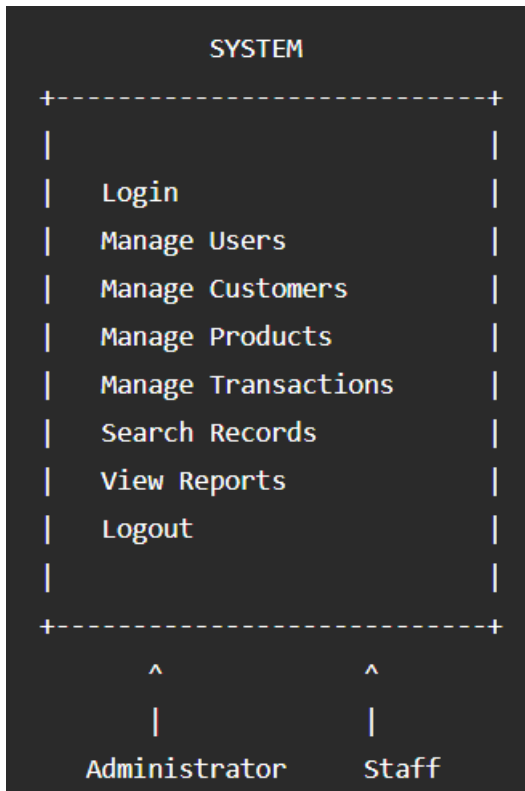
Example:



28. USE CASE DIAGRAM

The project documentation should include a Use Case Diagram.

Example:



The actual use cases should correspond to the functions implemented in the application.

29. DATABASE NORMALIZATION

The database should avoid unnecessary duplication.

The design should generally follow normalization principles.

First Normal Form (1NF)

- Each field should contain a single value.
- Avoid multiple values in one field.

Incorrect:

Products = "Laptop, Mouse, Keyboard"

Better:

Store individual products in a product table and associate them with transaction details.

Second Normal Form (2NF)

Non-key attributes should depend on the whole primary key.

Third Normal Form (3NF)

Non-key attributes should depend only on the primary key and not on another non-key attribute.

30. DATABASE CONSTRAINTS

The database should use appropriate constraints.

Examples:

Primary Key

PRIMARY KEY

Foreign Key

FOREIGN KEY

Unique

UNIQUE

Not Null

NOT NULL

Default

DEFAULT

Constraints should be used where appropriate to protect data integrity.

31. ERROR HANDLING

The application must handle errors properly.

Examples:

- Invalid login
- Empty fields
- Invalid number
- Database connection failure
- Duplicate record
- Missing record
- Invalid foreign key
- SQL exception
- Unexpected application error

Example:

```
try {  
    // database operation  
} catch (SQLException e) {  
    JOptionPane.showMessageDialog(  
        null,  
        "Database error: " + e.getMessage()  
    );  
}
```

Users should receive understandable messages rather than raw application crashes.

32. USER-FRIENDLY REQUIREMENTS

The interface should:

- Use consistent colors.
- Use readable fonts.
- Align labels and input fields properly.
- Provide clear button names.
- Use confirmation dialogs for destructive actions.
- Display success/error messages.
- Prevent invalid input where possible.
- Provide navigation between forms.
- Keep the interface organized.

Avoid overcrowding the screen with unnecessary components.

33. SECURITY REQUIREMENTS

The system should provide basic security measures.

Recommended:

- Login authentication
- Role-based access where appropriate
- Password fields
- Restricted administrative functions
- Prepared statements
- Input validation
- Logout functionality

For a real-world system, passwords should not be stored as plain text; password hashing should be considered.

34. REPORTING REQUIREMENTS

The system should provide useful information to users.

Examples:

- List of customers
- List of products
- Transaction history
- Inventory status
- User list
- Daily transactions
- Monthly transactions
- Summary of records

Reports may initially be displayed using JTable.

35. REQUIRED DOCUMENTATION

The final project documentation should contain the following sections:

Chapter 1 – Introduction

Include:

1. Background of the Study
2. System Overview
3. Problem Statement
4. Objectives
5. Scope and Limitations
6. Significance of the Project
7. Definition of Terms

Chapter 2 – System Analysis and Design

Include:

1. Existing System
2. Proposed System
3. Functional Requirements
4. Non-Functional Requirements
5. System Users
6. Use Case Diagram
7. System Flowchart
8. Data Flow Diagram
9. Entity Relationship Diagram
10. Database Design

Chapter 3 – System Development

Include:

1. Development Tools
2. Java Swing
3. Java AWT
4. JDBC
5. Apache Derby
6. System Architecture
7. Program Structure
8. Database Connection
9. CRUD Implementation
10. GUI Implementation

Chapter 4 – Testing

Include:

1. Test Plan
2. Test Cases
3. Login Testing
4. Input Validation Testing
5. CRUD Testing
6. Database Testing
7. Search Testing
8. Error Handling Testing
9. User Acceptance Testing

Chapter 5 – Conclusion

Include:

1. Summary
2. Conclusions
3. Recommendations
4. Possible Future Improvements

36. NON-FUNCTIONAL REQUIREMENTS

The system should also define non-functional requirements.

Usability

The system should be easy for intended users to understand.

Performance

Database searches and common operations should complete within a reasonable amount of time.

Reliability

The system should handle errors without unexpectedly terminating.

Maintainability

The code should be organized into logical classes and packages.

Security

Access to protected functionality should require appropriate authentication.

Data Integrity

The database should prevent invalid relationships and inappropriate duplicate data.

37. TESTING TABLE

The documentation should contain a test-case table similar to this:

Test ID	Function	Test Input	Expected Result	Actual Result	Status
T001	Login	Correct credentials	Dashboard opens		
T002	Login	Wrong password	Error message		
T003	Add	Valid record	Record saved		
T004	Add	Empty required field	Validation message		
T005	Update	Modified record	Record updated		
T006	Delete	Existing record	Confirmation and deletion		
T007	Search	Existing keyword	Matching records shown		
T008	Search	Invalid keyword	No matching records		

38. MINIMUM DATABASE OPERATIONS

The completed project should demonstrate at least:

INSERT
SELECT
UPDATE
DELETE

It should also demonstrate:

WHERE
ORDER BY
JOIN

Additional SQL operations may be included where appropriate.

39. MINIMUM JAVA FEATURES

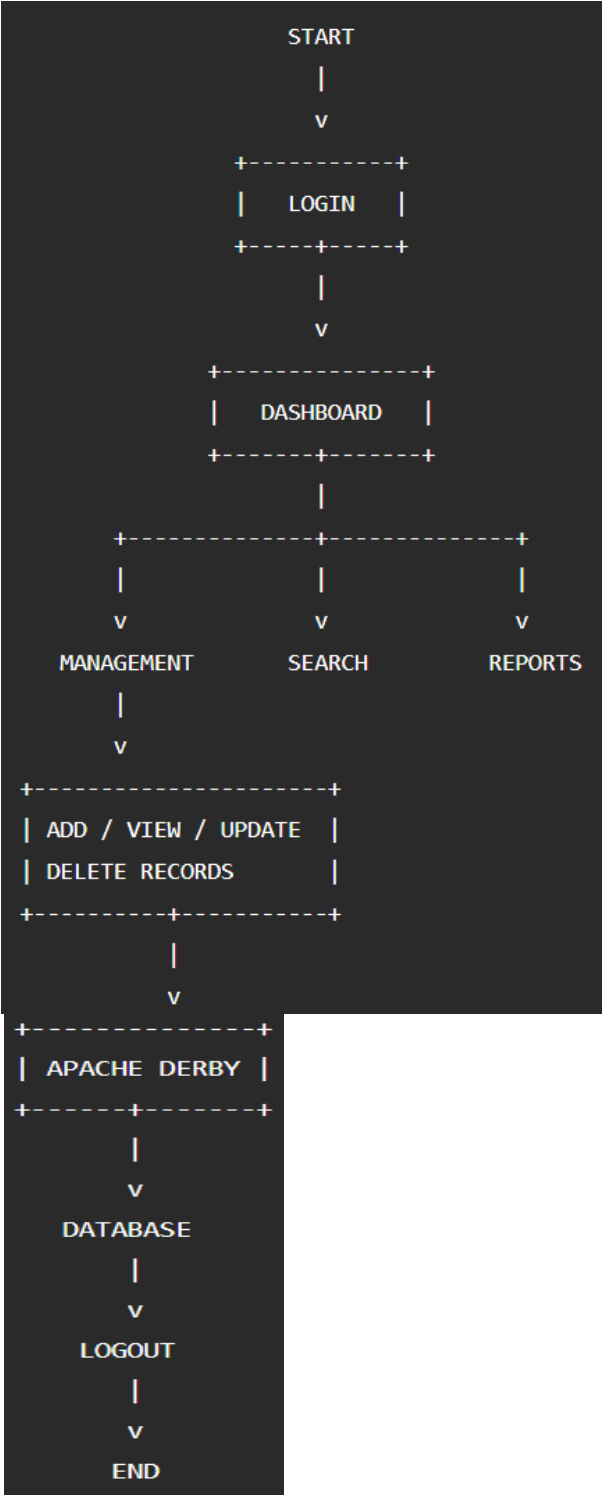
The project should demonstrate:

- Classes and objects
- Encapsulation
- Constructors
- Methods
- Exception handling
- Event handling
- GUI design
- Database connectivity
- SQL execution
- CRUD operations

- Tables/collections where appropriate
- Appropriate use of packages

40. EXPECTED FINAL SYSTEM

The completed application should have a flow similar to:



41. REQUIRED DIAGRAMS CHECKLIST

The final submission should contain, where applicable:

- System Architecture Diagram
- Use Case Diagram
- System Flowchart
- Data Flow Diagram (DFD)
- Entity Relationship Diagram (ERD)
- Database/Table Relationship Diagram
- Class Diagram
- GUI Navigation Diagram

42. REQUIRED DATABASE DOCUMENTATION

For every table, provide:

1. Table name
2. Purpose
3. Field name
4. Data type
5. Field size where applicable
6. Primary key
7. Foreign key
8. Nullable/not-null status
9. Default value where applicable
10. Description

Example:

PRODUCTS

Field	Type	PK	FK	Null	Description
product_id	INTEGER	Yes	No	No	Product identifier
category_id	INTEGER	No	Yes	No	Product category
product_name	VARCHAR(100)	No	No	No	Product name
price	DECIMAL(10,2)	No	No	No	Product price
quantity	INTEGER	No	No	No	Available quantity
status	VARCHAR(20)	No	No	No	Product status

43. PROJECT PRESENTATION REQUIREMENTS

During the presentation or defense, students should be able to demonstrate:

A. Login

Show successful and unsuccessful login attempts.

B. Database Connection

Demonstrate that the Java application connects to Derby.

C. Add

Create a new record.

D. View

Display database records in a JTable.

E. Search

Search for an existing record.

F. Update

Modify a record and demonstrate that the database changes.

G. Delete

Delete a record after confirmation.

H. Relationships

Demonstrate that related tables are connected through foreign keys.

I. Error Handling

Demonstrate how the system handles invalid input.

J. Reports

Display at least one useful report or summary.

44. FINAL SUBMISSION CHECKLIST

Before submitting the project, verify the following:

Software

- Java project runs successfully.
- Swing interface is functional.
- AWT functionality is demonstrated.
- JDBC connection works.
- Apache Derby database works.
- All required tables exist.
- Primary keys are defined.
- Foreign keys are defined.
- Relationships work correctly.
- CRUD operations work.
- Search works.
- Input validation works.
- Error handling works.
- Login works if required.
- Logout works if required.

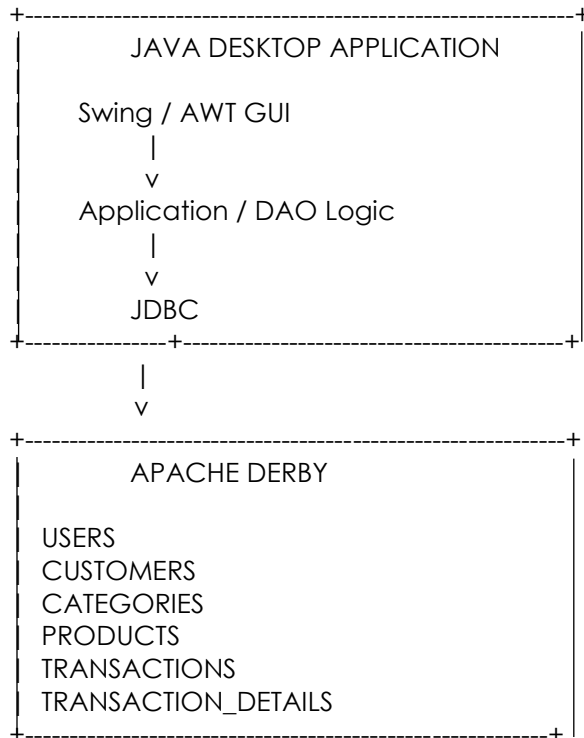
Documentation

- Introduction
- Background
- Problem Statement
- Objectives
- Scope and Limitations
- Significance
- System Requirements
- Functional Requirements
- Non-Functional Requirements
- Use Case Diagram
- Flowchart
- DFD
- ERD
- Table Relationship Diagram
- Class Diagram
- Database Dictionary
- SQL statements
- System screenshots
- Testing documentation
- Conclusion
- Recommendations

45. GENERAL PROJECT REQUIREMENT

The project must demonstrate that the student understands the relationship between the **GUI, Java application logic, JDBC, and database.**

The expected architecture is:



The final system should not only demonstrate a working interface. It should demonstrate a **complete information-management solution**, where the Java GUI communicates with the Derby database through JDBC, performs validated CRUD operations, and maintains properly related database records.

The specific entities and tables may be modified according to the selected system topic, but the same principles of **GUI design, database normalization, JDBC connectivity, relational database design, validation, CRUD operations, and documentation** should be maintained.